

IT SERVICE MANAGEMENT (TICKET) UYGULAMASI TASARIM RAPORU

HAZIRLAYAN

Enes AKICI B241210380

SAKARYA Mart 2026

İçindekiler

1. GİRİŞ (INTRODUCTION)	3
1.1. Dokümanın Amacı	3
1.2. Kapsam	3
1.3. Referanslar	3
1.4. Tanımlar ve Kısaltmalar	4
2. SİSTEM MİMARİSİ (HIGH-LEVEL DESIGN)	5
2.1. Mimari Yaklaşım	5
2.2. Sistem Bileşenleri ve Etkileşimleri.....	5
2.3. Kullanılan Teknolojiler ve Gerekçeleri.....	6
2.4. Dağıtım Mimarisi (Deployment Diagram)	7
3. DETAYLI TASARIM (LOW-LEVEL DESIGN).....	9
3.1. Veritabanı Tasarımı	9
3.2. Sınıf Diyagramı (Domain Model)	11
4. DAVRANIŞSAL TASARIM (BEHAVIORAL DESIGN).....	13
4.1. Sıra Diyagramları (Sequence Diagrams).....	13
4.2. Durum Diyagramı (State Diagram).....	15
5. KULLANICI ARAYÜZÜ TASARIMI.....	17
5.1. Kimlik Doğrulama Ekranları (Keycloak)	17
5.2. Müşteri Portalı Ekranları.....	18
5.3. Destek Uzmanı (Agent) Ekranları	19
5.4. Yönetici (Manager) Ekranları	20
6. VERİ YÖNETİMİ VE GÜVENLİK TASARIMI	21
6.1. Veri Saklama ve Bütünlük	21
6.2. Kimlik Doğrulama ve Yetkilendirme	21
6.3. Veri Gizliliği	21
6.4. API ve İletişim Güvenliği.....	21
7. ENTEGRASYON VE API TASARIMI	22
7.1. Ticket Yönetimi Uç Noktaları.....	22
7.2. Yorum ve İletişim Uç Noktaları	22
7.3. Efor Yönetimi (Worklog) Uç Noktaları	23
7.4. Bildirim Uç Noktaları	23
7.5. Webhook Uç Noktaları (Dış Sistemler İçin)	23
8. TEKNİK ALTYAPI VE DAĞITIM	25
8.1. Geliştirme Ortamı.....	25
8.2. Derleme ve Paketleme.....	25
8.3. Dağıtım (Docker Compose ile).....	25
9. TEST STRATEJİSİ.....	26
9.1. Birim Testleri (Unit Tests).....	26
9.2. Entegrasyon Testleri	26
9.3. Kabul Testleri (Acceptance Tests).....	26

1. GİRİŞ (INTRODUCTION)

1.1. Dokümanın Amacı

Bu doküman, **IT Service Management (Ticket) Uygulamasının** yazılım tasarımını detaylandırmaktadır. Analiz raporunda belirlenen fonksiyonel ve fonksiyonel olmayan gereksinimler doğrultusunda, sistemin nasıl gerçekleştirileceğine dair mimari kararları, bileşen tasarımlarını, veritabanı modelini, davranışsal modelleri ve teknik altyapıyı açıklamaktadır. Bu doküman, **geliştirme sürecinde yol gösterici olmayı** ve sistemin sürdürülebilir, ölçeklenebilir ve test edilebilir bir şekilde inşa edilmesini sağlamayı hedefler.

1.2. Kapsam

Bu tasarım dokümanı;

- Sistem mimarisini (katmanlı mimari, bileşenler, teknoloji yığını),
- Veritabanı tasarımını (ER diyagramı, tablo şemaları),
- Sınıf ve bileşen diyagramlarını,
- Davranışsal modelleri (sequence, state diyagramları),
- Kullanıcı arayüzü tasarımını (ekran akışları, bileşen yapısı),
- Güvenlik ve veri yönetimi stratejilerini,
- Entegrasyon detaylarını (jBPM, Keycloak, bildirim servisi),
- Dağıtım mimarisi ve CI/CD sürecini,
- Test stratejisini

kapsar. Doküman, MVP (Minimum Viable Product) sürümüne odaklanmakta olup, ileriki sürümlerde eklenmesi planlanan özellikler "Gelecek Çalışmalar" bölümünde not edilmiştir.

1.3. Referanslar

- IT Service Management (Ticket) Uygulaması Analiz Raporu – Enes ÇAKICI, Ocak 2026
- SAU-2026: IT Service Management (Ticket) Uygulaması Genel Teknik Gereksinimler Dokümanı

1.4. Tanımlar ve Kısaltmalar

Analiz raporunda tanımlanan terim ve kısaltmalar (Ticket, SLA, Worklog, jBPM, Internal/External Not, vb.) aynen geçerlidir. Ek olarak tasarım sürecinde kullanılacak bazı teknik terimler aşağıda listelenmiştir:

Terim/Kısaltma	Anlamı
JWT	JSON Web Token, yetkilendirme için kullanılan token formatı
RBAC	Role-Based Access Control, Rol Tabanlı Erişim Kontrolü
ERD	Entity-Relationship Diagram, Varlık-İlişki Diyagramı
API	Application Programming Interface, Uygulama Programlama Arayüzü
REST	Representational State Transfer, web servisleri için mimari stil
DTO	Data Transfer Object, katmanlar arası veri taşıma nesnesi
ORM	Object-Relational Mapping, nesne-ilişkisel eşleme (Hibernate)
CORS	Cross-Origin Resource Sharing, kaynak paylaşım politikası

2. SİSTEM MİMARİSİ (HIGH-LEVEL DESIGN)

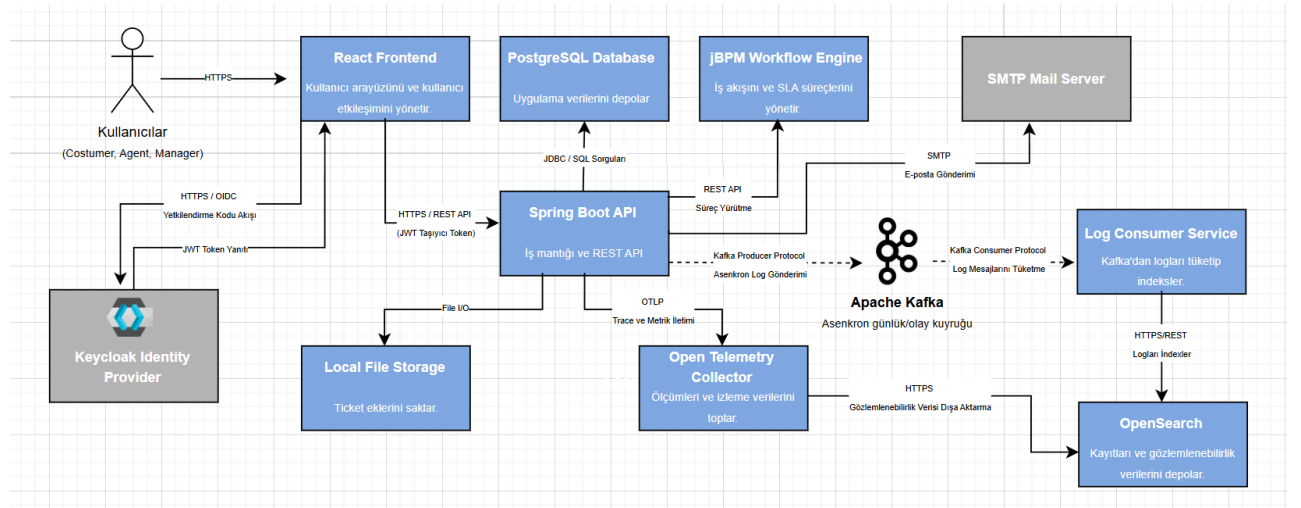
2.1. Mimari Yaklaşım

Sistem, **katmanlı mimari** prensibine göre tasarlanmıştır. Bu yaklaşım, endişelerin ayrılması (separation of concerns) ilkesini benimseyerek her katmanın belirli bir sorumluluğa sahip olmasını sağlar. Başlıca katmanlar:

- **Sunum Katmanı (Presentation Layer):** React ile geliştirilmiş web arayüzü. Kullanıcı etkileşimlerini yönetir ve backend ile REST API üzerinden haberleşir.
- **İş Mantığı Katmanı (Business Logic Layer):** Spring Boot ile geliştirilmiş backend uygulaması. Controller (API giriş noktaları), Service (iş kuralları) ve Repository (veritabanı erişimi) alt katmanlarından oluşur.
- **Veri Erişim Katmanı (Data Access Layer):** PostgreSQL veritabanı ve Hibernate ORM. Verilerin kalıcı olarak saklanması ve yönetilmesini sağlar.
- **Entegrasyon Katmanı (Integration Layer):** Harici sistemlerle (Keycloak, jBPM) iletişimi yönetir. Bu bileşenler ayrı servisler olarak çalışır ve backend üzerinden entegre edilir.

2.2. Sistem Bileşenleri ve Etkileşimleri

Sistem, frontend (React), backend (Spring Boot) ve destekleyici altyapı bileşenlerinin (PostgreSQL, Keycloak, jBPM, Kafka, OpenSearch) birbiriyle entegre çalıştığı dağıtık bir yapıdan oluşmaktadır. Bileşenlerin genel etkileşimi ve hiyerarşisi aşağıdaki kapsayıcı diyagramında (Container Diagram) gösterilmiştir:



Şekil 1. Container diagram

2.3. Kullanılan Teknolojiler ve Gerekçeleri

Sistem mimarisini oluşturan teknolojiler, MVP süresi ve endüstri standartları göz önüne alınarak seçilmiştir:

Teknoloji	Seçim Gerekçesi
Java / Spring Boot	Olgun, güvenli ve yaygın kullanılan bir backend framework'ü. Katmanlı mimari desteği, geniş kütüphane ekosistemi ve kolay test edilebilirlik sağlar.
React	Modern, component tabanlı bir frontend kütüphanesi. Hızlı UI geliştirme, geniş topluluk desteği ve yeniden kullanılabilir bileşenler sunar.
PostgreSQL	ACID uyumlu, güvenilir ve açık kaynaklı ilişkisel veritabanı. JSON desteği ile esnek veri saklama imkanı sunar (ileride ihtiyaç duyulursa).
Keycloak	Açık kaynaklı, hazır kimlik ve erişim yönetimi çözümü. SSO, 2FA, LDAP entegrasyonu gibi özellikleri kutu içinde sunar, sıfırdan geliştirme maliyetini düşürür.
jBPM	Açık kaynaklı iş akış motoru. Kod dışında tanımlanmış süreçler (BPMN) ile SLA timer, durum geçişleri gibi karmaşık iş mantığını yönetmeyi kolaylaştırır.
Apache Kafka	Yüksek hacimli log mesajlarının asenkron ve güvenilir şekilde iletilmesi için dağıtık mesaj kuyruğu. Logların kaybolmadan merkezi sisteme aktarılmasını sağlar ve sistemler arası gevşek bağlılık sunar.
Log Consumer Service	Kafka'dan log mesajlarını tüketen, gerekli dönüşümleri yaparak OpenSearch'e indeksleyen özel bir mikroservistir. Bu sayede log akışı yönetilebilir ve ölçeklenebilir hale gelir.
OpenSearch	Log ve metrik verilerinin merkezi olarak toplanması, indekslenmesi ve analiz edilmesi için açık kaynaklı bir arama ve analiz motoru. Gerçek zamanlı sorgulama, görselleştirme ve dashboard imkanı sunar.
OpenTelemetry	Uygulama performans metriklerinin (istek sayısı, yanıt süresi, hata oranı) ve dağıtık izleme (trace) verilerinin toplanması için açık kaynaklı bir framework. Toplanan veriler OpenSearch'e aktararak görselleştirilebilir.
Docker & Docker Compose	Uygulamanın tüm bileşenlerini (backend, frontend, veritabanı, Keycloak, jBPM, Kafka, OpenSearch vb.) konteynerize ederek bağımlılıkları izole eder, kurulumu ve dağıtımını kolaylaştırır. Docker Compose ile tüm servisler tek bir komutla ayağa kaldırılabilir.
SMTP Mail Server (Spring Mail)	E-posta bildirimlerini göndermek için Spring Boot'un sağladığı mail desteği kullanılacaktır. Gmail, Outlook gibi harici SMTP sunucuları veya yerel bir mail sunucusu ile entegre çalışabilir.
Local File Storage	Ticket ekleri (log, ekran görüntüsü) sunucunun yerel dosya sisteminde belirlenen bir dizinde saklanır. Veritabanında dosyanın yolu (path) ve meta bilgileri tutulur. İleride bulut depolama (AWS S3 vb.) seçeneğine geçilebilir.

2.4. Dağıtım Mimarisi (Deployment Diagram)

MVP (Minimum Viable Product) sürümünde sistemin tüm bileşenleri, **tek bir sunucu (veya geliştirme makinesi)** üzerinde **Docker konteynerleri** olarak çalışacaktır. Konteyner orkestrasyonu için **Docker Compose** kullanılacaktır.

2.4.1. Docker Compose ile Çalıştırılacak Servisler

Servis Adı	Teknoloji	Açıklama
postgres-db	PostgreSQL	Veritabanı sunucusu. Tüm uygulama verilerini (kullanıcı, ticket, yorum, dosya meta verileri) saklar. jBPM de kendi süreç verileri için bu veritabanını kullanır.
keycloak	Keycloak	Kimlik ve erişim yönetimi sunucusu. Kullanıcı doğrulama, JWT token üretimi ve RBAC yetkilendirmesini sağlar.
jbpm-server	jBPM	İş akış motoru. Ticket yaşam döngüsü ve SLA timer'larını yönetir.
backend-app	Spring Boot	REST API'leri sunar, iş mantığını yürütür, diğer servislerle (Keycloak, jBPM, Kafka, OpenTelemetry) iletişim kurar.
frontend-app	React + Nginx	Kullanıcı arayüzü. Nginx üzerinden statik dosyaları sunar. Geliştirme ortamında React kendi sunucusuyla çalışırken, production benzeri dağıtımda Nginx ile sunulur.
kafka	Apache Kafka	Mesaj kuyruğu. Backend'den gelen log mesajlarını toplar ve Log Consumer Service'e iletir. Zookeeper ile birlikte çalışır.
log-consumer	Spring Boot / Python	Kafka'dan log mesajlarını okuyan, işleyen ve OpenSearch'e indeksleyen mikroservis.
opensearch	OpenSearch	Log ve metrik verilerinin merkezi olarak depolandığı, indekslendiği ve sorgulanabildiği arama ve analiz motoru.
opensearch-dashboards	OpenSearch Dashboards	OpenSearch içindeki verileri görselleştirmek, sorgulamak ve yönetim panelleri (dashboard) oluşturmak için web tabanlı arayüz.
otel-collector	OpenTelemetry Collector	Backend'den OTLP protokolü ile gönderilen trace ve metrik verilerini toplar, işler ve OpenSearch'e aktarır.
smtp-server	MailHog (geliştirme) veya harici SMTP	Geliştirme ortamında MailHog kullanılabilir; gerçek ortamda Gmail, Outlook vb. SMTP sunucusu konfigüre edilir.

2.4.2. Ağ Yapısı ve İletişim

- Tüm konteynerlar **aynı Docker ağı** üzerinde çalışır. Bu sayede servis isimleri üzerinden birbirlerine erişebilirler.
- **Dış dünyaya açılan servisler:**
 - **Frontend (Nginx):** Kullanıcıların web uygulamasına erişimi.
 - **Backend API:** Dış istemcilerin (frontend, Postman vb.) API'ye erişimi.
 - **Keycloak:** Kullanıcıların login ekranına erişebilmesi ve JWT token alabilmesi için.
 - **OpenSearch Dashboards:** Yöneticilerin log ve metrikleri izleyebilmesi, dashboard oluşturabilmesi için.
- **Dış dünyaya kapalı olan servisler:**
 - PostgreSQL, jBPM, Kafka, Log Consumer, OpenSearch, OpenTelemetry Collector, SMTP sunucusu. Bu servislere güvenlik amacıyla sadece Docker iç ağındaki diğer servisler erişebilir.

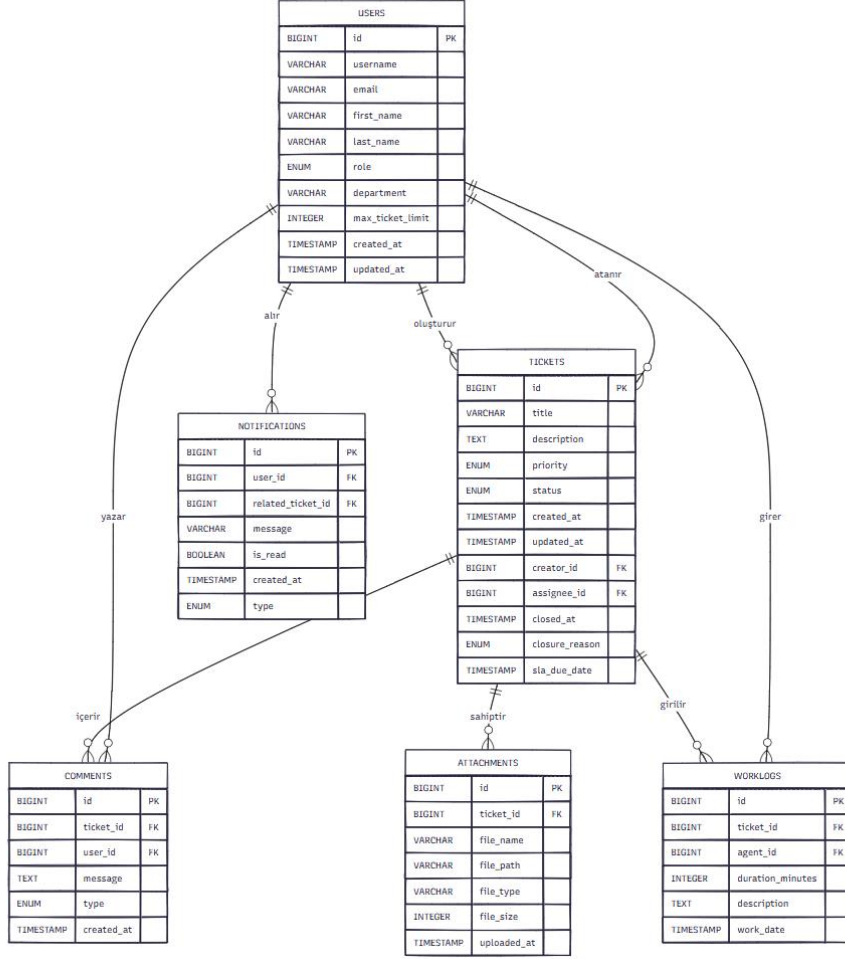
2.4.3. Veri Kalıcılığı (Persistence)

- **PostgreSQL, OpenSearch ve Kafka** gibi kalıcı veri saklayan servisler için **Docker volume'leri** kullanılır. Bu sayede konteyner yeniden başlatıldığında veri kaybolmaz.
- Ticket ek dosyaları, sunucunun yerel dosya sisteminde belirlenen bir dizinde saklanır ve ilgili konteynera volume olarak mount edilir.
- jBPM ve Keycloak, ana verilerini PostgreSQL'de saklasalar da, kendi konfigürasyon dosyaları (örneğin Keycloak temaları, jBPM süreç tanımları) için ayrı Docker volume'lerine ihtiyaç duyar. Bu sayede konteyner yeniden başlatıldığında özelleştirmeler korunur.

3. DETAYLI TASARIM (LOW-LEVEL DESIGN)

3.1. Veritabanı Tasarımı

3.1.1. Varlık-İlişki Diyagramı (ERD)



Şekil 2. Varlık-İlişki Diyagramı (ERD)

3.1.2. Tablo Şemaları

Aşağıdaki tablo şemaları, sistemdeki tüm varlıkları ve aralarındaki ilişkileri tanımlar. **NOT:** PK = Primary Key, FK = Foreign Key, ENUM = Önceden tanımlı sabit değerler.

users (Kullanıcılar)

Alan Adı	Veri Tipi	Kısıt	Açıklama
id	BIGINT	PK	Benzersiz kullanıcı numarası (otomatik artan)
username	VARCHAR(50)	UNIQUE, NOT NULL	Kullanıcı adı (Keycloak ile senkronize)
email	VARCHAR(100)	UNIQUE, NOT NULL	E-posta adresi
first_name	VARCHAR(50)	NOT NULL	Ad
last_name	VARCHAR(50)	NOT NULL	Soyad
role	ENUM('CUSTOMER','AGENT','MANAGER')	NOT NULL	Kullanıcı rolü
department	VARCHAR(50)	NULL	Departman (sadece AGENT/MANAGER için)

max_ticket_limit	INTEGER	DEFAULT 5	AGENT rolü için üzerine alabileceği maksimum açık ticket sayısı
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Kayıt oluşturulma zamanı
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Son güncelleme zamanı (trigger ile otomatik güncellenir)

tickets (Ticketlar)

Alan Adı	Veri Tipi	Kısıt	Açıklama
id	BIGINT	PK	Benzersiz ticket numarası
title	VARCHAR(200)	NOT NULL	Ticket başlığı
description	TEXT	NOT NULL	Detaylı açıklama
priority	ENUM('HIGH','MEDIUM','LOW')	NOT NULL	Öncelik
status	ENUM('NEW','IN_PROGRESS','WAITING_FOR_CUSTOMER','RESOLVED','CLOSED')	NOT NULL	Ticket durumu
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Açılma zamanı
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Son güncelleme
creator_id	BIGINT	FK → users.id, NOT NULL	Ticket'i açan kullanıcı (müşteri)
assignee_id	BIGINT	FK → users.id, NULL	Atanan agent (atanmamışsa NULL)
closed_at	TIMESTAMP	NULL	Kapanma zamanı (kapandıysa dolu)
closure_reason	ENUM('SOLVED','CUSTOMER_APPROVED','INVALID','NO_RESPONSE','DUPLICATE')	NULL	Kapatma nedeni (kapandıysa dolu)
sla_due_date	TIMESTAMP	NULL	Ticket'in SLA'ya göre çözülmesi gereken son tarih. (In Progress olduğunda hesaplanır ve yazılır.)

comments (Yorumlar)

Alan Adı	Veri Tipi	Kısıt	Açıklama
id	BIGINT	PK	Benzersiz yorum numarası
ticket_id	BIGINT	FK → tickets.id, NOT NULL	Hangi ticketa ait
user_id	BIGINT	FK → users.id, NOT NULL	Yorumu yazan kullanıcı
message	TEXT	NOT NULL	Yorum içeriği
type	ENUM('INTERNAL','EXTERNAL')	NOT NULL	İç (sadece ekip) veya dış (müşteri de görür) yorum
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Yorum zamanı

attachments (Dosya Ekleri)

Alan Adı	Veri Tipi	Kısıt	Açıklama
id	BIGINT	PK	Benzersiz dosya numarası
ticket_id	BIGINT	FK → tickets.id, NOT NULL	Hangi ticketa ait
uploaded_by	BIGINT	FK → users.id, NOT NULL	Dosyayı yükleyen kullanıcı (müşteri veya agent)
file_name	VARCHAR(255)	NOT NULL	Orijinal dosya adı (örnek: "ekran.png")
file_path	VARCHAR(500)	NOT NULL	Sunucudaki fiziksel yol (örnek: "/uploads/ticket/123/ekran.png")
file_type	VARCHAR(50)	NOT NULL	MIME tipi (örnek: "image/png", "application/pdf")
file_size	INTEGER	NOT NULL	Dosya boyutu (byte cinsinden)
uploaded_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Yükleme zamanı

worklogs (Efor Kayıtları)

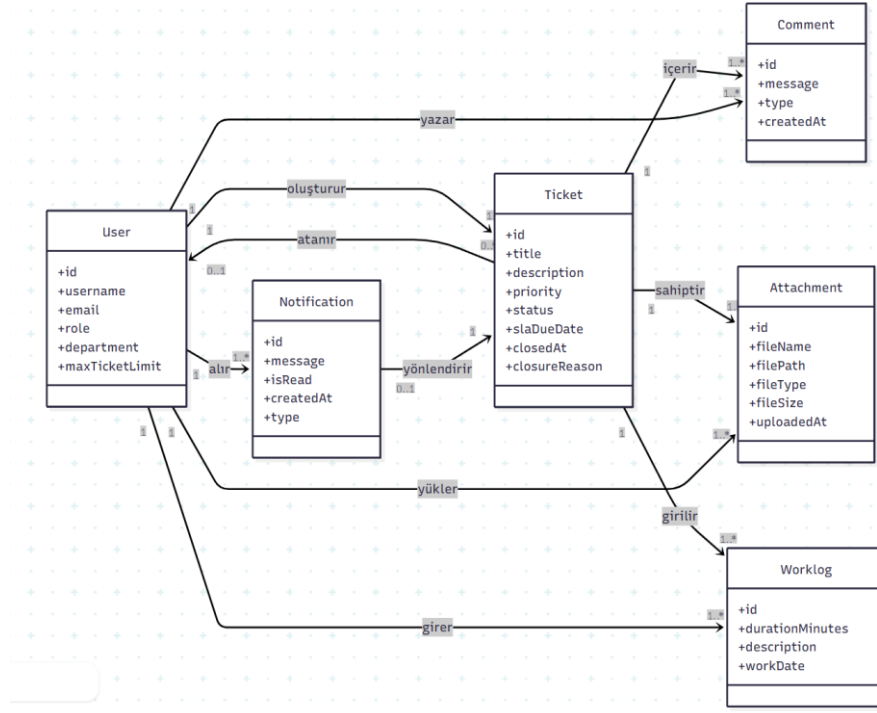
Alan Adı	Veri Tipi	Kısıt	Açıklama
id	BIGINT	PK	Benzersiz worklog numarası
ticket_id	BIGINT	FK → tickets.id, NOT NULL	Hangi ticketa ait
agent_id	BIGINT	FK → users.id, NOT NULL	Eforu giren agent (sadece AGENT rolü)
duration_minutes	INTEGER	NOT NULL	Harcanan süre (dakika)
description	TEXT	NULL	Yapılan işin açıklaması (opsiyonel)
work_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Çalışmanın yapıldığı zaman

notifications (Bildirimler)

Alan Adı	Veri Tipi	Kısıt	Açıklama
id	BIGINT	PK	Benzersiz bildirim numarası
user_id	BIGINT	FK → users.id, NOT NULL	Bildirimin gideceği kullanıcı
related_ticket_id	BIGINT	FK → tickets.id, NULL	Bildirime tıklandığında gidilecek ticket ID'si (ticket ile ilgili bildirimler için)
message	VARCHAR(500)	NOT NULL	Bildirim mesajı (örnek: "Yeni ticket atandı: #123")
is_read	BOOLEAN	DEFAULT FALSE	Okundu bilgisi
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Bildirim oluşturulma zamanı
type	ENUM('TICKET_CREATED','TICKET_ASSIGNED','STATUS_CHANGED','COMMENT_ADDED','SLA_WARNING','SLA_BREACHED')	NOT NULL	Bildirim tipi (ileride filtreleme için)

3.2. Sınıf Diyagramı (Domain Model)

Bu projede, kodun sürekli evrimleşen yapısı (Controller, Service, Repository katmanları) yerine, sistemin çekirdek iş mantığını temsil eden Etki Alanı Modeli (Domain Model) sınıf diyagramı tercih edilmiştir. Bu diyagram, sistemdeki temel varlıkları (Entity) ve aralarındaki nesne tabanlı (OOP) ilişkileri ve kardinaliteleri (multiplicity) gösterir. Metotlar (getter/setter veya iş metotları) bu diyagramda gösterilmemiştir; odak noktası varlıklar ve ilişkilerdir.



Şekil 3. Domain Model

Sınıflar ve Özellikleri

Sınıf Adı	Özellikler (Fields)
User	id, username, email, role, department, maxTicketLimit
Ticket	id, title, description, priority, status, slaDueDate, closedAt, closureReason
Comment	id, message, type (INTERNAL/EXTERNAL), createdAt
Attachment	id, fileName, filePath, fileType, fileSize, uploadedAt
Worklog	id, durationMinutes, description, workDate
Notification	id, message, isRead, createdAt, type

Sınıf İlişkileri ve Kardinaliteler (Multiplicity)

İlişki	Kardinalite	Açıklama
User (creator) – Ticket	1 .. * (bir kullanıcı birden çok ticket açar)	Bir ticket'ın bir oluřturucusu (creator) vardır.
User (assignee) – Ticket	0 .. * (bir agent'a sıfır veya daha fazla ticket atanabilir)	Bir ticket en fazla bir agent'a atanır (atanmamıř olabilir).
Ticket – Comment	1 .. * (bir ticket birden çok yorum içerebilir)	Her yorum bir ticketa aittir.
User – Comment	0 .. * (Ticket ilk açıldıđında yorum olmayabilir)	Her yorum bir kullanıcı tarafından yazılır.
Ticket – Attachment	1 .. * (bir ticket birden çok dosya ekine sahip olabilir)	Her dosya eki bir ticketa aittir.
User – Attachment	1 .. * (bir kullanıcı birden çok dosya yükleyebilir)	Dosyayı yükleyen kullanıcı (uploaded_by) bilgisi vardır.
Ticket – Worklog	0 .. * (Henüz efor girilmemiř olabilir)	Her worklog bir ticketa aittir.
User – Worklog	1 .. * (bir agent birden çok worklog girebilir)	Worklog'u giren agent bilgisi vardır.
User – Notification	1 .. * (bir kullanıcının birden çok bildirimi olabilir)	Her bildirim bir kullanıcıya aittir.
Notification – Ticket	0 .. 1 (bildirim bir ticket ile ilişkili olabilir veya olmayabilir)	Bildirime tıklandıđında ilgili ticketa yönlendirme için.

4. DAVRANIŞSAL TASARIM (BEHAVIORAL DESIGN)

Bu bölümde, sistemin statik mimarisinin ötesine geçilerek, bileşenlerin (React, Spring Boot, Keycloak, jBPM, Kafka) kritik iş senaryolarında birbirleriyle olan dinamik ve zamana bağlı etkileşimleri modellenmiştir.

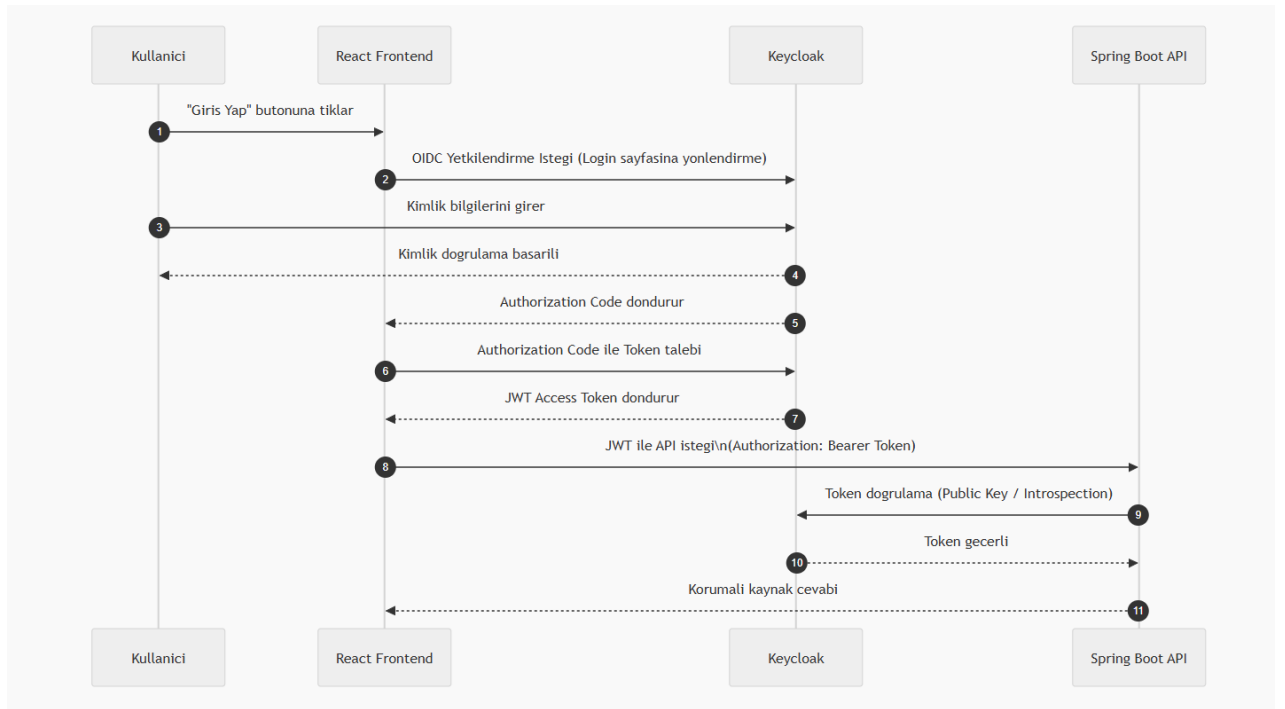
4.1. Sıra Diyagramları (Sequence Diagrams)

Sistemdeki en kritik uçtan uca iş akışları, senkron ve asenkron iletişim modelleri dikkate alınarak aşağıdaki sıra diyagramları ile ifade edilmiştir.

4.1.1. Sisteme Giriş ve Yetkilendirme (OIDC Akışı) Senaryosu

Bu senaryo, kullanıcının sisteme güvenli bir şekilde giriş yapmasını ve backend servislerine erişim için gerekli olan JWT token'ı alma sürecini gösterir.

1. Kullanıcı arayüze (React) girer ve "Giriş Yap" butonuna tıklar.
2. React, kullanıcıyı Keycloak'un login sayfasına yönlendirir (OIDC Authorization Code Flow).
3. Kullanıcı kimlik bilgilerini girer ve Keycloak doğrulamayı yapar.
4. Keycloak, React'e güvenli bir JWT (Access Token) döner.
5. React, bu token'ı kullanarak Spring Boot API'sine güvenli bir istek atar.

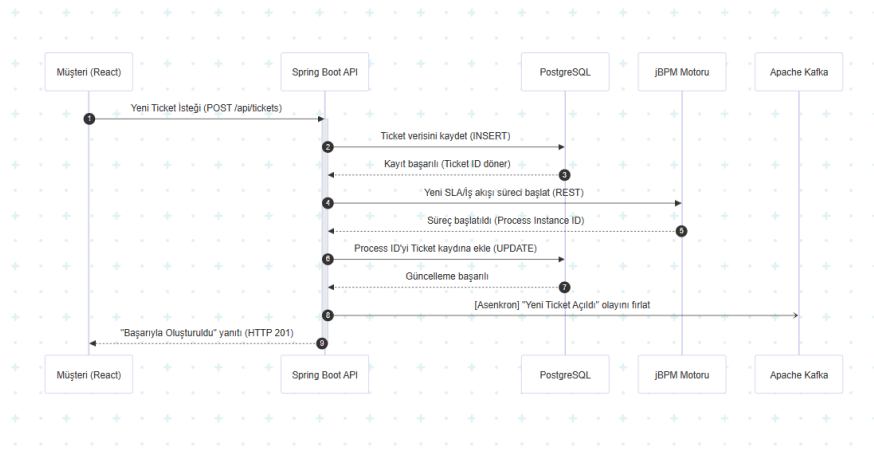


Şekil 4. Sisteme Giriş ve Yetkilendirme (OIDC Akışı) Diyagramı

4.1.2. Yeni Ticket Oluşturma Senaryosu (Kafka ve jBPM Tetiklenmesi)

Bu senaryo, sistemin ana işlevidir. Bir ticket oluşturulduğunda veritabanı, iş akış motoru (jBPM) ve loglama (Kafka) mekanizmalarının nasıl birlikte çalıştığını gösterir.

1. Müşteri, React üzerinden "Yeni Ticket" formunu doldurur ve API'ye POST isteği atar.
2. Spring Boot API, ticket verisini PostgreSQL veritabanına kaydeder.
3. Spring Boot, jBPM REST API'sine istek atarak bu ticket için yeni bir iş akışı süreci (Process Instance) başlatır.
4. Spring Boot, kullanıcının beklemesini engellemek için arka planda (Asenkron) Kafka'ya "Yeni ticket oluşturuldu" log mesajını gönderir.
5. API, React arayüzüne "Başarılı" yanıtını döner.

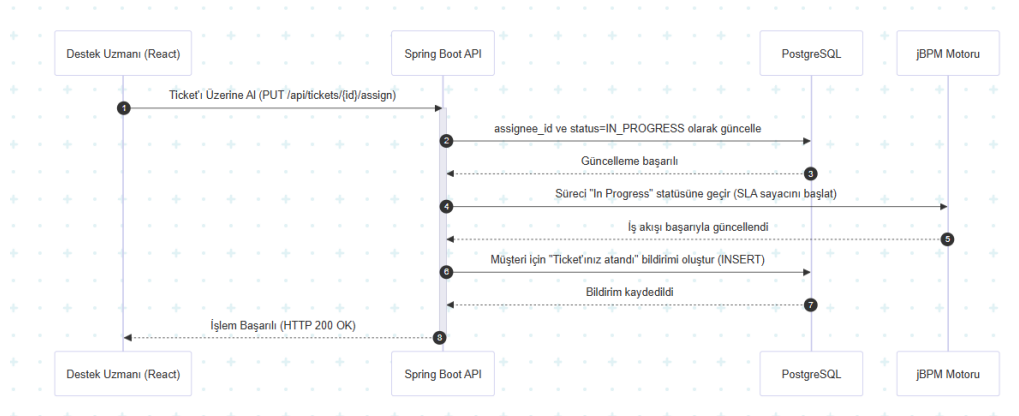


Şekil 5. Yeni Ticket Oluşturma Diyagramı

4.1.3. Ticket Atama (Assign) Senaryosu

Sisteme düşen bir talebin, bir destek uzmanı (Agent) tarafından üzerine alınması sürecidir.

1. Agent, React arayüzünden "Bana Ata" butonuna tıklar.
2. Spring Boot API, PostgreSQL'de ilgili ticket'ın assignee_id alanını günceller.
3. Spring Boot, jBPM'e istek atarak ticket durumunu "In Progress" (İşlemde) statüsüne geçirir.
4. Müşteriye bilgi vermek amacıyla sistem içi bir bildirim (Notification) kaydı oluşturulur.

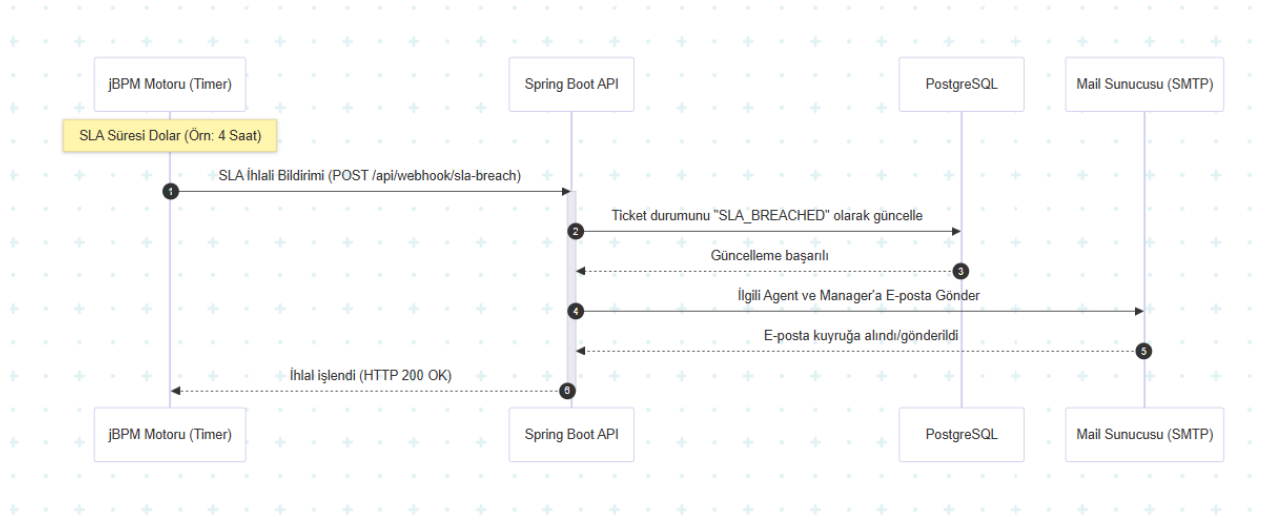


Şekil 6. Ticket Atama (Assign) Diyagramı

4.1.4. SLA Zamanlayıcı ve İhlal Bildirimi Senaryosu

jBPM iş akış motorunun zaman bazlı (Timer) olarak sistemi nasıl dışarıdan tetiklediğini gösteren senaryodur.

1. jBPM içindeki SLA sayacı dolar (Örn: 4 saat boyunca işlem yapılmadı).
2. jBPM, Spring Boot API'sindeki bir webhook/REST ucuna "SLA İhlali" isteği atar.
3. Spring Boot, veritabanında ticket'a SLA ihlal işaretini (flag) koyar.
4. Spring Boot, SMTP Sunucusu üzerinden ilgili Agent ve Manager'a e-posta gönderir.

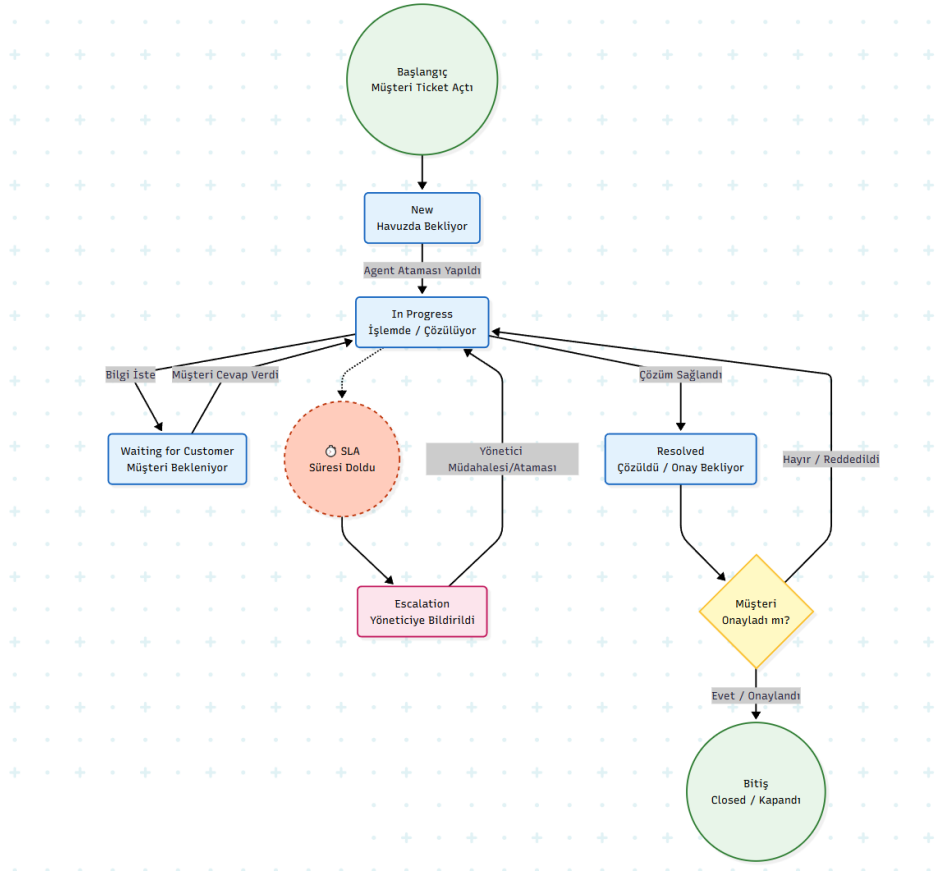


Şekil 7. SLA Zamanlayıcı ve İhlal Bildirim Diyagramı

4.2. Durum Diyagramı (State Diagram)

Ticket'ın yaşam döngüsü boyunca geçtiği durumlar ve geçiş kuralları:

- **Havuzda Bekliyor (New):** Müşteri ticket oluşturduğunda sistemin atadığı ilk durumdur. Talep, uygun bir destek uzmanı (Agent) atanana kadar havuzda bekler.
- **İşlemde / Çözülüyor (In Progress):** Havuzdaki talep bir Agent üzerine atandığında aktif işlem aşamasına geçilir. (Not: SLA sayacı bu durumda çalışır. Süre aşılsa sistem durumu değiştirmez, ancak bilet üzerine 'SLA İhlali' bayrağı ekleyerek yöneticiye bildirim atar - Bkz: Senaryo 4.1.4).
- **Müşteri Bekleniyor (Waiting for Customer):** Agent, sorunun çözümü için müşteriden ek log, görsel veya bilgi talep ettiğinde geçilen durumdur. Müşteri yanıt verdiğinde süreç tekrar "İşlemde" durumuna döner.
- **Çözüldü / Onay Bekliyor (Resolved):** Agent sorunu çözdüğünü beyan eder. Bu aşamada nihai karar müşterininidir.
- **Karar Noktası ve Bitiş (Closed):** Müşteri sunulan çözümü onaylarsa ticket "Kapandı" (Closed) statüsüne geçer ve yaşam döngüsü biter. Müşteri çözümü reddederse, bilet tekrar "İşlemde" statüsüne çekilerek efor süreci yeniden başlatılır.



Şekil 8. Durum Diyagramı

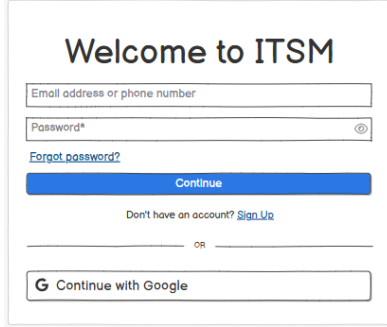
5. KULLANICI ARAYÜZÜ TASARIMI

Bu bölümde, analiz fazında hazırlanan ekran taslakları (wireframe) rol bazlı olarak sunulmuştur.

5.1. Kimlik Doğrulama Ekranları (Keycloak)

Kimlik doğrulama işlemleri Keycloak sunucusu tarafından sağlanır. Bu ekranlar React uygulamasından bağımsızdır.

- **Giriş Ekranı:** Kullanıcı adı ve şifre ile sisteme giriş.
- **Kayıt Ol Ekranı:** Yeni kullanıcı kaydı.
- **2FA Ekranı:** Agent/Manager rolleri için çift aşamalı doğrulama.



Welcome to ITSM

Email address or phone number

Password*

[Forgot password?](#)

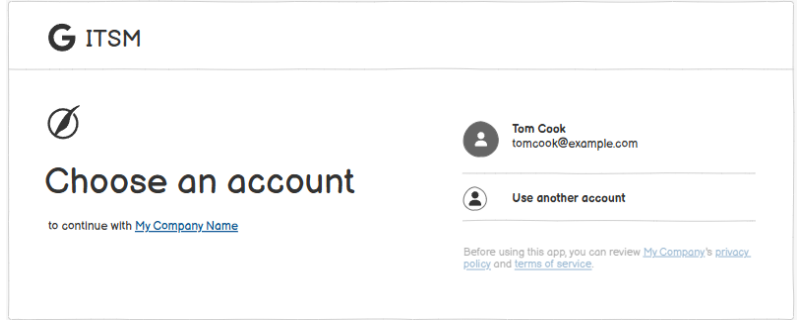
Continue

Don't have an account? [Sign Up](#)

OR

Continue with Google

Şekil 9. Giriş Ekranı



G ITSM

Choose an account

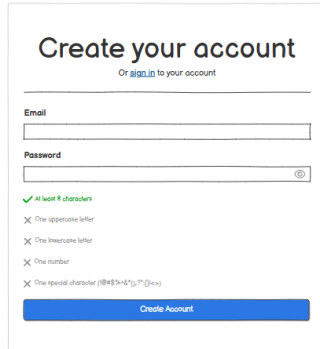
to continue with [My Company Name](#)

Tom Cook
tomcook@example.com

Use another account

Before using this app, you can review [My Company's privacy policy](#) and [terms of service](#).

Şekil 10. Google ile Devam Et Ekranı



Create your account

Or [sign in](#) to your account

Email

Password

✓ At least 8 characters

✗ One uppercase letter

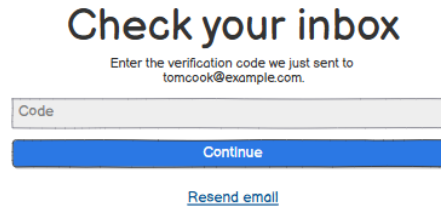
✗ One lowercase letter

✗ One number

✗ One special character ([@#%*~!<"/>?(){}~])

Create Account

Şekil 11. Kayıt Ol Ekranı



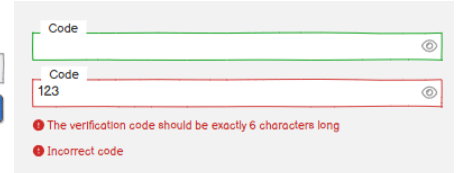
Check your inbox

Enter the verification code we just sent to tomcook@example.com.

Code

Continue

[Resend email](#)



Code

Code

123

• The verification code should be exactly 6 characters long

• Incorrect code

Şekil 12. Çift Aşamalı Doğrulama Ekranı

5.2. Müşteri Portali Ekranları

Müşterilerin kendi ticketlarını yönettiği self-servis ekranlardır.

- **Ticket Listesi:** Müşterinin açtığı ticketların listesi (filtreleme ve arama ile).
- **Yeni Ticket Açma:** Başlık, açıklama, öncelik ve dosya eki ile ticket oluşturma.
- **Ticket Detay:** Ticket geçmişi (timeline) ve yorum yapma.

[Anasayfa](#) > Taleplerim Yeni Ticket Aç

Statü

Öncelik

Tarih Aralığı

Ticket ID veya Başlık ile Ara

New, In Progress...

Yüksek, Orta, Düşük

Başlangıç - Bitiş

Ticket ID	Problem Title	Status	Priority	Created Date	Updated Date
1001	AWS S3 Bağlantı Zaman Aşımı	New	Yüksek	2024-05-20	2024-05-21
1002	Senkronizasyon Hatası	In Progress	Yüksek	2024-05-20	2024-05-21
1003	Frontend Auth-Token Yenileme	Resolved	Düşük	2024-05-20	2024-05-21
1004	Docker Container Başlatma Hatası	Waiting for Customer	Yüksek	2024-05-20	2024-05-21
1005	SQL Query Optimizasyonu	Closed	Orta	2024-05-20	2024-05-21
1006	SSL Handshake Başarısız	New	Düşük	2024-05-20	2024-05-21
1007	CI/CD Pipeline Deployment Hatası	In Progress	Yüksek	2024-05-20	2024-05-21
1008	Mobil API JSON Parse Hatası	Resolved	Düşük	2024-05-20	2024-05-21
1009	SMTP Mail Sunucusu Yanıt Vermiyor	Waiting for Customer	Yüksek	2024-05-20	2024-05-21

Şekil 13. Müşteri Ticket Listesi Ekranı

[Anasayfa](#) > Yeni Ticket Aç

Başlık*

Açıklama*

Ürün Seçimi*

Öncelik Seçimi

Dosya Yükleme

Ürün 1, Ürün 2

☒ Yüksek (Kritik Sorun, Acil Müdahale Gerekir)
☐ Orta (Önemli Sorun, Planlı Çözüm)
☐ Düşük (Geliştirme Talebi, Minör Hata)

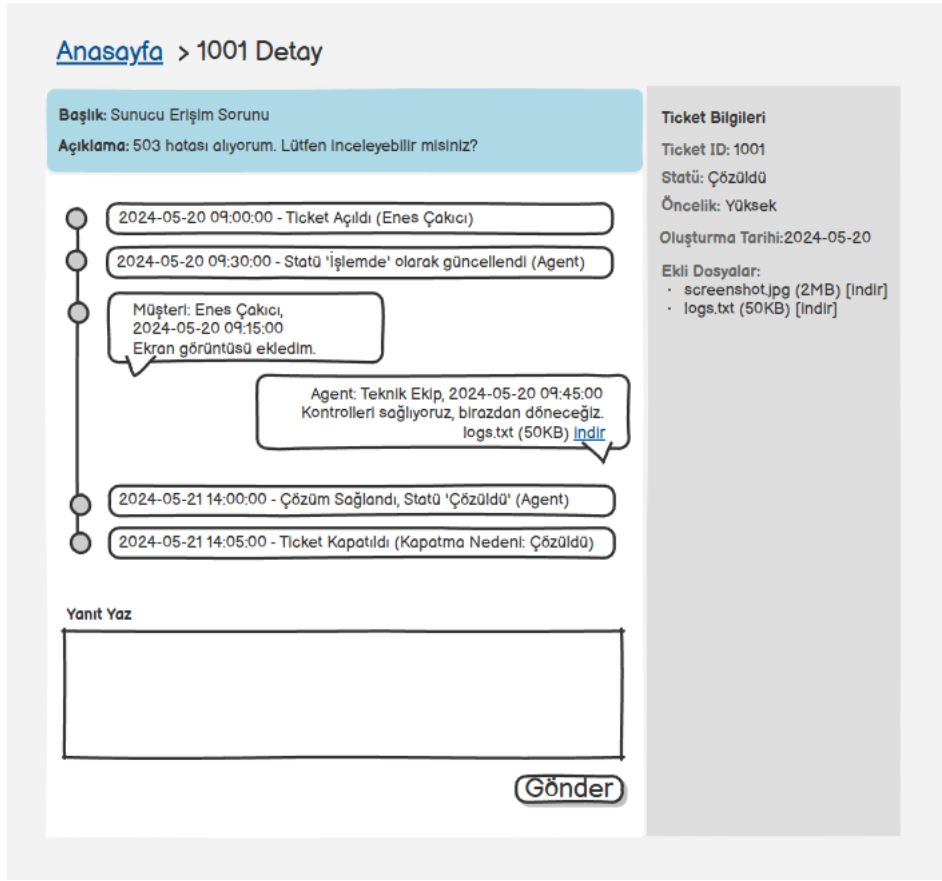
Choose files to Upload

or drag and drop them her

Gönder

İptal

Şekil 14. Yeni Ticket Açma Ekranı



Şekil 15. Müşteri Ticket Detay Ekranı

5.3. Destek Uzmanı (Agent) Ekranları

Agent'ların ticket operasyonlarını yürüttüğü ekranlardır.

- Ticket Havuzu:** Atanmamış ticketlar, SLA sayaçları, "Üzerine Al" butonu.
- Ticket Detay:** Müşteri detayları, durum değiştirme, internal yorum, worklog girişi, devir işlemi.

Ticket Havuzu / Liste Ekranı (Agent)							
Statü	Öncelik	Atanan Kişi	SLA Durumu	Tarih Aralığı	Ticket ID veya Başlık ile Ara		
New, In Progress...	Yüksek, Orta, Düşük	Atanmamış, Bana Ata...	İhlal Var, Risk Var, Nor...	Başlangıç - Bitiş			
Ticket ID	Başlık	Müşteri	Statü	Öncelik	SLA Kalan	Atanan Kişi	Aksiyon
1001	AWS S3 Bağlantı Zaman Aşımı	Tech Corp	New	Yüksek	30 Dk (İhlal)	Atanmamış	Üzerine Al
1002	Senkronizasyon Hatası	Global A.Ş.	In Progress	Yüksek	2 Saat (Risk)	Bana Atanmış	Detay
1003	Frontend Auth-Token Yenileme	Startup X	Resolved	Düşük	-	Bana Atanmış	Detay
1005	SQL Query Optimizasyonu	Tech Corp	Closed	Orta	-	Bana Atanmış	Detay
1006	SSL Handshake Başarısız	Global A.Ş.	New	Düşük	1 Gün	Atanmamış	Üzerine Al
1007	CI/CD Pipeline Deployment Hatası	Müşteri B	In Progress	Yüksek	4 Saat	Bana Atanmış	Detay
1009	SMTP Mail Sunucusu Yanıt Vermiyor	Tech Corp	Waiting	Yüksek	Duraklatıldı	Bana Atanmış	Detay

Şekil 16. Agent Ticket Havuzu / Liste Ekranı

Başlık: Sunucu Erişim Sorunu

Açıklama: 503 hatası alıyorum. Lütfen inceleyebilir misiniz?

2024-05-20 09:00:00 - Ticket Açıldı (Enes Çakıcı)

2024-05-20 09:30:00 - Statü "İşlemde" olarak güncellendi (Agent)

Müşteri: Enes Çakıcı, 2024-05-20 09:15:00
Ekran görüntüsü ekledim.

Agent: Teknik Ekip, 2024-05-20 09:45:00
Kontrolleri sağlıyoruz, birazdan döneceğiz.
logs.txt (50KB) [İndir](#)

Internal Note: Agent: Teknik Ekip, 2024-05-20 09:35:00
Loglarda yüksek trafik görüyorum, DB bağlantılarını kontrol edeceğim.

2024-05-21 14:00:00 - Çözüm Sağlandı, Statü "Çözüldü" (Agent)

2024-05-21 14:05:00 - Ticket Kapatıldı (Kapatma Nedeni: Çözüldü)

External (Müşteriye Yaz)

Internal (Ekip İçi Not)

Gönder

Ticket Bilgileri
Ticket ID: 1001
Statü: Çözüldü
Öncelik: Yüksek
Oluşturma Tarihi: 2024-05-20
Ekli Dosyalar:
- screenshot.jpg (2MB) [İndir](#)
- logs.txt (50KB) [İndir](#)

Aksiyonlar
Statü: **New, In Progress...**
Worklog Formu
Harcanan Efor(Saat/Dk)
Açıklama
[Ekle](#)

Devir
Kişi Seç [Devret](#)

Kapatma
Çözüm Kodu
Çözüldü, Müşteri onayı alındı [✓](#)

Şekil 17. Agent Ticket Detay Ekranı

5.4. Yönetici (Manager) Ekranları

- Dashboard:** Ticket istatistikleri, SLA ihlalleri, agent performansı.

Anasayfa > Dashboard

Tarih Aralığı: Son 30 Gün

Açık Ticketler
142

SLA İhlali
3

Risk Altındakiler
5

Ort. Çözüm Süresi
2.4 Saat

Statü Dağılımı (Aktif İş Yükü)

- In Progress
- New
- Waiting for Customer
- Resolved

Aylara/Haftalara Göre Gelen-Çözülen

Hafta	Gelen	Çözülen
Hafta 1	55	75
Hafta 2	85	75
Hafta 3	85	55
Hafta 4	65	85

Kritik Eşik / SLA Riski Olan Talepler

Ticket ID	Başlık	Öncelik	SLA Durumu	Alan Kişi
1001	AWS S3 Bağlantı Zaman Aşımı	Yüksek	İHLAL (30 dk gecikti)	Atanmamış Detaya Git
1004	Docker Container Başlatma Hatası	Yüksek	RISK (45 dk kaldı)	Mert Çelik Detaya Git
1007	CI/CD Pipeline Deployment Hatası	Yüksek	RISK (2 saat kaldı)	Enes Çakıcı Detaya Git

Şekil 18. Manager Dashboard

6. VERİ YÖNETİMİ VE GÜVENLİK TASARIMI

Bu bölümde, sistemdeki verilerin nasıl saklandığı, korunduğu ve kullanıcı yetkilendirmesinin nasıl yapıldığı açıklanmaktadır.

6.1. Veri Saklama ve Bütünlük

- Tüm ilişkisel veriler (kullanıcılar, ticketlar, yorumlar) **PostgreSQL** veritabanında saklanır.
- Veritabanı tabloları normalize edilmiş olup, foreign key kısıtları ile veri bütünlüğü sağlanır.
- Ticket ekleri (dosyalar) veritabanında değil, sunucunun yerel dosya sisteminde saklanır; veritabanında sadece dosya yolu tutulur.

6.2. Kimlik Doğrulama ve Yetkilendirme

- Kimlik doğrulama işlemleri **Keycloak** sunucusu tarafından yönetilir. Kullanıcı şifreleri uygulama veritabanında saklanmaz.
- React frontend, Keycloak ile **OIDC Authorization Code Flow + PKCE** kullanarak iletişim kurar. Başarılı giriş sonrası **JWT** alınır.
- JWT içindeki rol bilgisi (CUSTOMER, AGENT, MANAGER) ile **rol tabanlı yetkilendirme (RBAC)** uygulanır.
- Spring Boot backend, gelen JWT'leri Keycloak'ın public key'i ile doğrular ve Spring Security ile method seviyesinde yetkilendirme yapar.

6.3. Veri Gizliliği

- Yorumlar **internal** (sadece ekip görür) ve **external** (müşteri de görür) olarak ayrılmıştır. Bu ayırım veritabanında type alanı ile kontrol edilir.
- Hassas veriler (şifre vb.) uygulama tarafında hiçbir şekilde saklanmaz; tamamen Keycloak tarafından yönetilir.

6.4. API ve İletişim Güvenliği

- Tüm dış iletişim **HTTPS** ile şifrelenir.
- Backend API'leri **CORS** ile sadece frontend domain'inden gelen isteklere izin verecek şekilde yapılandırılmıştır.
- Rate limiting MVP'de uygulanmayacak, ancak ileride eklenmesi planlanmaktadır.

7. ENTEGRASYON VE API TASARIMI

Sistemde frontend (React) ve backend (Spring Boot) arasındaki iletişim RESTful prensiplere göre tasarlanmıştır. Tüm API uç noktaları JSON formatında veri alışverişi yapar ve Keycloak üzerinden alınan JWT token ile yetkilendirilir.

7.1. Ticket Yönetimi Uç Noktaları

HTTP Metodu	Uç Nokta (Endpoint)	Açıklama	Yetkilendirme (Rol)
POST	/api/tickets	Yeni bir ticket oluşturur ve jBPM sürecini başlatır.	CUSTOMER, AGENT, MANAGER
GET	/api/tickets	Ticket'ları listeler. Müşteri sadece kendi ticket'larını, AGENT ve MANAGER tüm ticket'ları görür.	CUSTOMER, AGENT, MANAGER
GET	/api/tickets/{id}	Belirtilen ticket'ın detaylarını getirir.	CUSTOMER (kendi ticket'ı), AGENT, MANAGER
PUT	/api/tickets/{id}/assign	Ticket'ı bir AGENT'a atar veya mevcut atamayı değiştirir.	AGENT, MANAGER
PUT	/api/tickets/{id}/transfer	Ticket'ı başka bir AGENT'a devreder (devir nedeni ile birlikte).	AGENT, MANAGER
PATCH	/api/tickets/{id}/status	Ticket durumunu günceller ve jBPM'e bildirir.	AGENT, MANAGER

7.2. Yorum ve İletişim Uç Noktaları

HTTP Metodu	Uç Nokta (Endpoint)	Açıklama	Yetkilendirme (Rol)
GET	/api/tickets/{ticketId}/comments	Bir ticket'a ait yorumları getirir. Internal yorumlar müşteriden gizlenir.	CUSTOMER (kendi ticket'ı), AGENT, MANAGER
POST	/api/tickets/{ticketId}/comments	Bir ticket'a yeni yorum ekler. Yorum tipi (INTERNAL/EXTERNAL) body'de belirtilir.	CUSTOMER (sadece EXTERNAL), AGENT, MANAGER (her iki tip)

7.3. Efor Yönetimi (Worklog) Uç Noktaları

HTTP Metodu	Uç Nokta (Endpoint)	Açıklama	Yetkilendirme (Rol)
POST	/api/tickets/{ticketId}/worklogs	Bir ticket'a ait efor kaydı ekler.	AGENT
GET	/api/worklogs/reports	AGENT'ların efor raporlarını getirir (tarih aralığı, agent bazlı filtreleme ile).	MANAGER

7.4. Bildirim Uç Noktaları

HTTP Metodu	Uç Nokta (Endpoint)	Açıklama	Yetkilendirme (Rol)
GET	/api/notifications	Giriş yapmış kullanıcının bildirimlerini getirir.	CUSTOMER, AGENT, MANAGER
PATCH	/api/notifications/{id}/read	Belirtilen bildirimi okundu olarak işaretler.	CUSTOMER, AGENT, MANAGER
PATCH	/api/notifications/read-all	Kullanıcının tüm bildirimlerini okundu olarak işaretler.	CUSTOMER, AGENT, MANAGER

7.5. Webhook Uç Noktaları (Dış Sistemler İçin)

HTTP Metodu	Uç Nokta (Endpoint)	Açıklama	Yetkilendirme (Rol)
POST	/api/webhook/jbpm/sla-breach	jbPM tarafından SLA ihlali durumunda çağrılır. İç sistemdir, dışarıya kapalıdır.	INTERNAL (IP veya API key ile korunur)

7.6. Dosya Yönetimi (Attachment) Uç Noktaları

HTTP Metodu	Uç Nokta (Endpoint)	Açıklama	Yetkilendirme (Rol)
POST	/api/tickets/{ticketId}/attachments	Bir ticket'a dosya ekler. İstek tipi multipart/form-data olmalıdır. Maksimum dosya sayısı (5) ve boyut (10 MB) kontrolleri burada yapılır.	CUSTOMER, AGENT, MANAGER
GET	/api/attachments/{id}	Yüklenmiş bir dosyayı indirmek/görüntülemek için kullanılır. Sadece o ticket'ı görmeye yetkisi olanlar indirebilir.	CUSTOMER (kendi ticket'ı), AGENT, MANAGER

7.7. Sistem Konfigürasyonu Uç Noktaları

HTTP Metodu	Uç Nokta (Endpoint)	Açıklama	Yetkilendirme (Rol)
PUT	/api/settings/agent-limit	Bir Agent'ın üzerine alabileceği maksimum eşzamanlı ticket limitini (Varsayılan: 5) günceller.	MANAGER

8. TEKNİK ALTYAPI VE DAĞITIM

Bu bölümde, geliştirme ortamları, derleme araçları ve sistemin canlı/test ortamına nasıl taşınacağı (Deployment) detaylandırılmıştır. Süreçlerin tamamı hızlı kurulum ve taşınabilirlik odağında tasarlanmıştır.

8.1. Geliştirme Ortamı

Projenin geliştirme aşamasında endüstri standardı araçlar kullanılmıştır:

- **IDE:** Backend (Spring Boot) için IntelliJ IDEA, Frontend (React) için Visual Studio Code tercih edilmiştir.
- **Çalışma Zamanı ve Bağımlılıklar:**
 - Backend için Java Development Kit (JDK 17 veya üzeri) kullanılmıştır.
 - Frontend geliştirme sürecinde ve paket yönetimi için Node.js kuruludur. (Canlı sunucuda Node.js çalıştırılmaz, sadece statik dosyalar sunulur.)
- **Lokal Servisler:** Geliştirme sırasında Keycloak, Kafka, jBPM, PostgreSQL, OpenSearch ve OpenTelemetry gibi altyapı bileşenleri, geliştirici makinesini kirletmemek adına Docker konteynerlerinde izole olarak çalıştırılmaktadır.

8.2. Derleme ve Paketleme

Kodların sunucuda çalıştırılmaya hazır hale getirilmesi için standart derleme araçları kullanılmıştır:

- **Backend (Spring Boot):** Maven ile kaynak kod yönetimi ve derleme yapılmıştır. `mvn clean package` komutu ile uygulama bağımsız bir `.jar` dosyasına dönüştürülür. Ardından bir `Dockerfile` ile bu JAR, sadece Java çalışma ortamı içeren hafif bir Docker imajına paketlenir.
- **Frontend (React):** `npm run build` komutu ile React kodu, tarayıcıda çalışacak optimize edilmiş statik dosyalara (HTML/JS/CSS) derlenir. Bu dosyalar, hızlı ve güvenli sunum için bir Nginx web sunucusu imajı içine yerleştirilir.

8.3. Dağıtım (Docker Compose ile)

MVP aşamasında karmaşık orkestrasyon araçları (Kubernetes gibi) yerine, tek sunuculu ve basit bir dağıtım yaklaşımı benimsenmiştir:

- **Versiyon Kontrol:** Tüm kodlar Git tabanlı bir platformda (GitHub/GitLab) saklanır.
- **Konteynerleştirme:** Sistem bileşenleri (postgres-db, keycloak, kafka, jbpn, opensearch, otel-collector, backend, frontend) tek bir `docker-compose.yml` dosyasında tanımlanmıştır.
- **Kurulum:** Sunucuda proje repostan çekilir ve `docker-compose up -d` komutu çalıştırılır. Tüm servisler, Docker Network üzerinde izole bir ağda dakikalar içinde ayağa kalkar. Bu yaklaşım, sistemin taşınabilirliğini sağlar ve ileride otomatik CI/CD süreçlerine geçişi kolaylaştıracak bir temel oluşturur.

9. TEST STRATEJİSİ

Bu bölümde, sistemin doğru ve güvenilir çalıştığından emin olmak için yapılacak testler açıklanmaktadır. MVP sürecinde testler, temel işlevleri ve sistemin birlikte çalışmasını kontrol edecek şekilde planlanmıştır.

9.1. Birim Testleri (Unit Tests)

Backend kodunun (Spring Boot) küçük parçalarının tek başına test edilmesidir. Özellikle:

- Servis katmanındaki iş kuralları (ticket oluşturma, durum geçişleri gibi)
- Yardımcı fonksiyonlar

Bu testler JUnit ve Mockito ile yazılacak ve geliştirme sırasında sık sık çalıştırılarak kodun temel işlevlerinin doğru olduğu kontrol edilecektir.

9.2. Entegrasyon Testleri

Sistemin farklı parçalarının birbiriyle uyumlu çalıştığını test etmek için:

- **API Testleri:** REST API uç noktalarının doğru çalıştığı, yetkilendirme (JWT) kurallarına uyduğu Postman veya benzeri araçlarla test edilecektir.
- **Veritabanı Testleri:** Repository katmanının gerçek bir veritabanına karşı doğru çalıştığı kontrol edilecektir.
- **Harici Servis Testleri:** Keycloak, jBPM ve Kafka gibi dış sistemlerle iletişimin temel düzeyde test edilmesi (MVP kısıtı nedeniyle sınırlı tutulacaktır).

9.3. Kabul Testleri (Acceptance Tests)

Sistemin, analiz raporunda belirlenen kabul kriterlerini karşıladığını doğrulamak için yapılacak testlerdir. Geliştirme tamamlandıktan sonra manuel olarak yürütülecektir. Öne çıkan senaryolar:

- **İş Yüğü Testi:** Bir agent 5'ten fazla açık ticket alamamalı.
- **SLA Testi:** Ticket "Müşteri Bekleniyor" durumundayken SLA süresi durmalı, tekrar başlatılınca kaldığı yerden devam etmeli.
- **SLA İhlal Testi:** Yüksek öncelikli ticket 4 saatte çözülmezse yöneticiye bildirim gitmeli.
- **Veri Gizliliği Testi:** Internal yorumlar müşteri tarafından görülmemeli.
- **Kapatma Nedeni Testi:** Ticket kapatılırken kapatma nedeni seçilmesi zorunlu olmalı.

10. SONUÇ VE GELECEK ÇALIŞMALAR

Bu tasarım raporunda, IT Service Management (Ticket) Uygulaması'nın MVP sürümüne yönelik kapsamlı bir yazılım mimarisi ortaya konmuştur. Sistem; modern teknolojiler (Spring Boot, React, PostgreSQL, Keycloak, jBPM) kullanılarak, ölçeklenebilir, güvenli ve bakımı kolay bir yapıda tasarlanmıştır.

Rapor kapsamında:

- Sistem mimarisi ve bileşenleri (Container Diagram)
- Veritabanı tasarımı (ERD ve tablo şemaları)
- Domain model sınıf diyagramı
- Davranışsal tasarım (sequence ve state diyagramları)
- Kullanıcı arayüzü taslakları
- Veri yönetimi ve güvenlik tasarımı
- API tasarımı ve entegrasyon detayları
- Teknik altyapı ve dağıtım planı
- Test stratejisi

detaylı olarak ele alınmıştır.

Gelecek Çalışmalar:

MVP sürümünün tamamlanmasının ardından sisteme eklenmesi planlanan özellikler şunlardır:

- **Mobil uygulama desteği:** React Native ile müşteri ve agent için temel işlevleri içeren mobil uygulama.
- **Müşteri memnuniyet anketi:** Ticket kapandıktan sonra müşteriye gönderilecek anket ile hizmet kalitesi ölçümü.
- **Gelişmiş raporlama ve dashboard:** Manager için daha detaylı grafikler, filtreleme ve export seçenekleri.
- **Etiketleme (tag) sistemi:** Ticketlara etiket atayarak kategorizasyon ve filtreleme kolaylığı.
- **Çalışma saatleri ve tatil günlerine göre SLA hesaplama:** İş takvimi entegrasyonu ile daha gerçekçi SLA süreleri.
- **Rate limiting ve gelişmiş API güvenlik önlemleri:** DDoS koruması ve istek sınırlandırma.
- **Yedekleme otomasyonu ve uzak depolama:** Veritabanı yedeklerinin otomatik alınması ve farklı lokasyonlarda saklanması.

Bu tasarım dokümanı, geliştirme sürecine yol gösterici olacak ve sistemin sürdürülebilir bir şekilde inşa edilmesini sağlayacaktır.